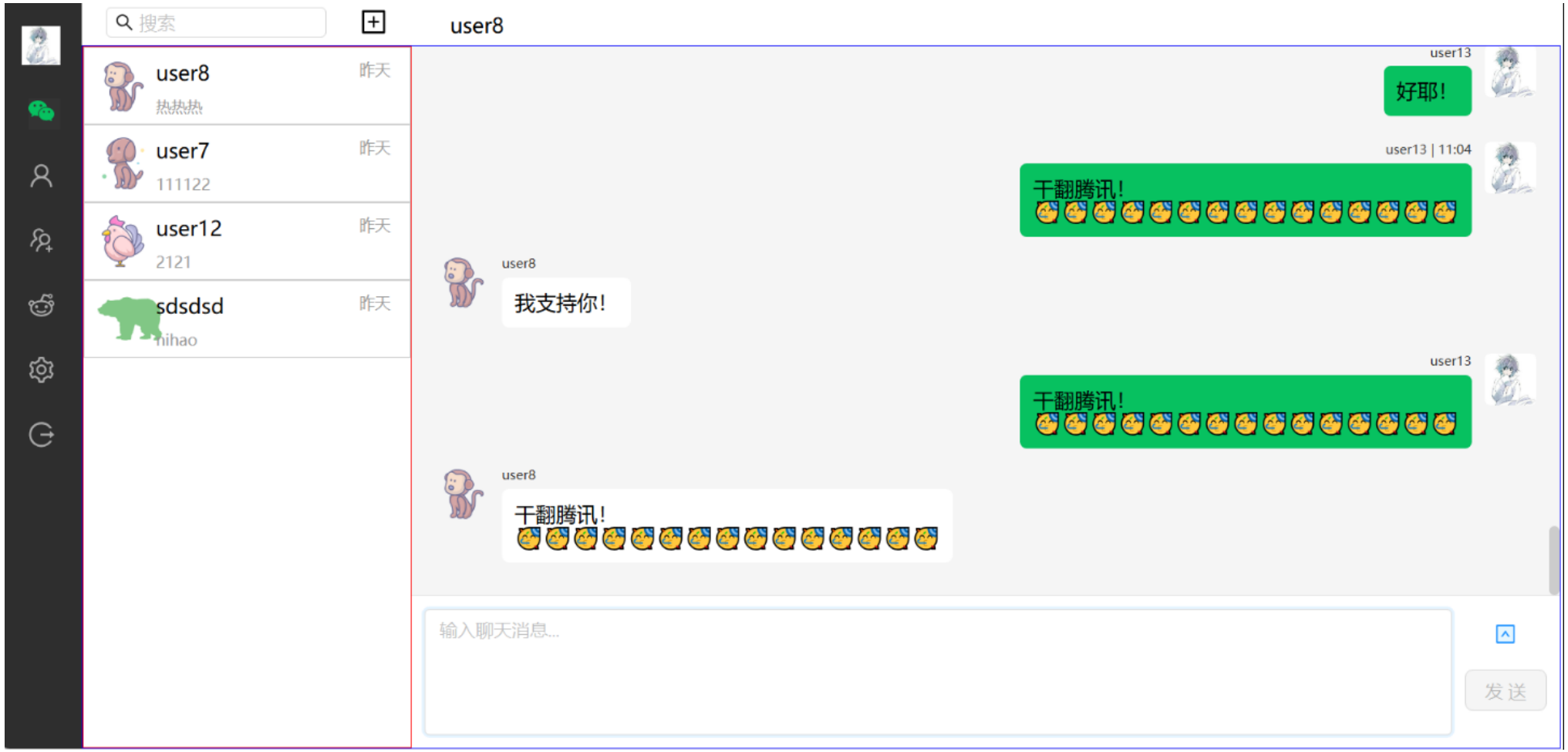
仿微信聊天软件



~~——也不知道当时哪个组想的这么难的破项目~~

仿微信聊天软件

随着移动通信的普及，社交软件在日常生活中扮演着越来越重要的角色。

为了满足用户对即时通讯的需求，开发一款仿微信的聊天软件，可以提升用户的沟通效率，增强用户体验。

这是一款能够为用户提供安全便捷，功能丰富的社交软件。

项目技术栈

前端：vue3、electron、ant-design、vite、pinia



后端：springboot、mybatis-plus、Maven



数据库：mysql



消息协议：WebSocket



第三方服务：百度翻译api



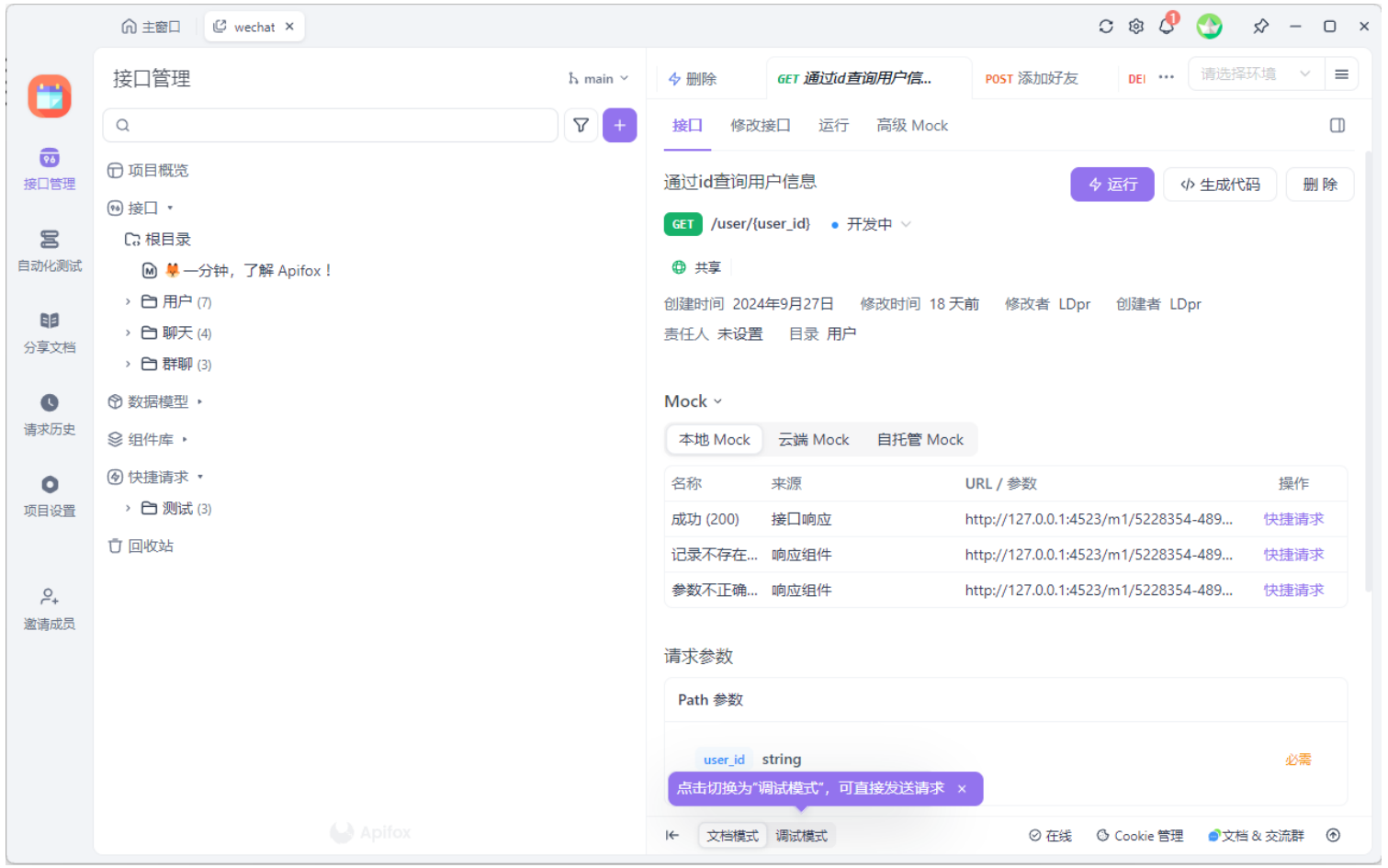
讯飞星火大模型



开发流程——对标企业级开发规范

APIFOX助力前后端并行

使用APIFOX进行数据模拟，在接口没有开发完成情况下，前端也能同步进行开发。



代码规范——避免💩代码

.env.development A

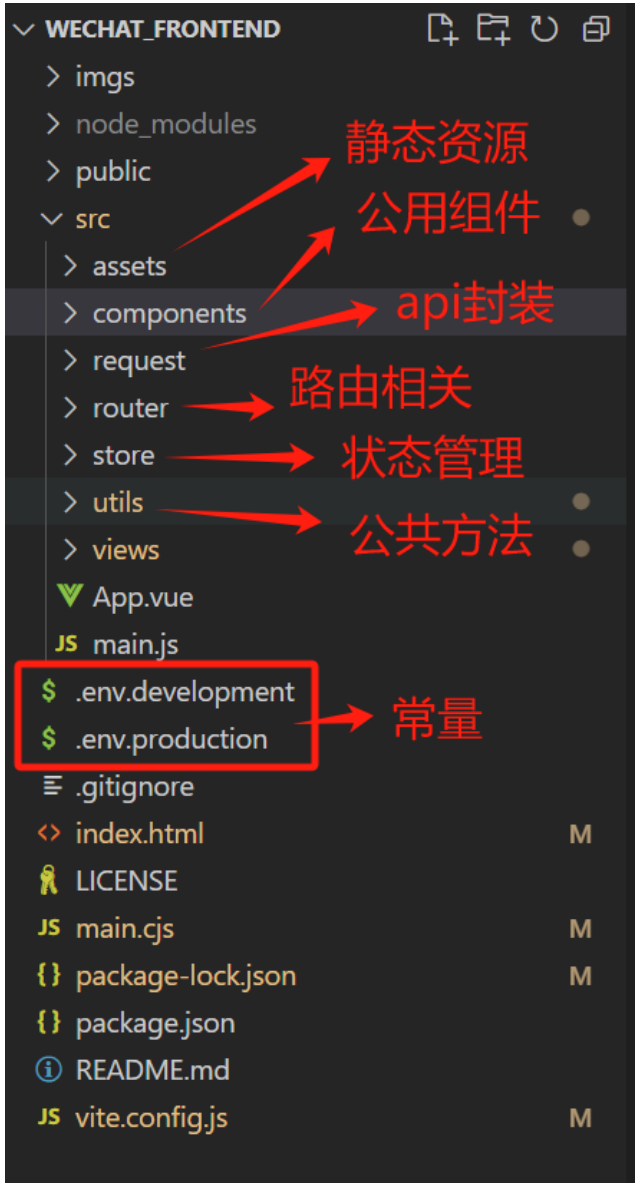
frontend > .env.development

```
# 百度翻译api_key
VITE_TRANSLATE_APP_ID=""
VITE_TRANSLATE_API_KEY=""
# 讯飞星火api_key
VITE_SPARKAI_APPID=""
VITE_SPARKAI_KEY=""
VITE_SPARKAI_SECRET=""

VITE_BASE_URL="/api"
VITE_WS_URL="ws://localhost:8084"
```

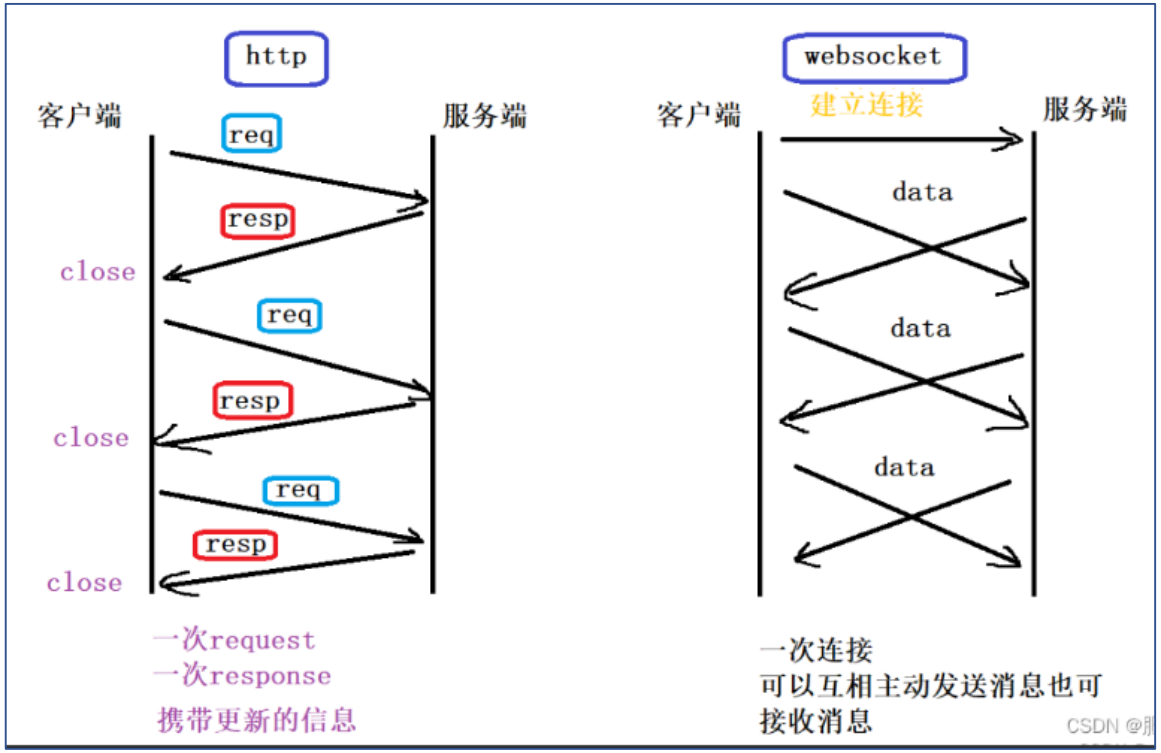
```
<script setup>
import { onMounted, ref, watchEffect } from 'vue';
import { message } from "ant-design-vue";
import CryptoJS from "crypto-js";
import { debounce } from "@/utils/index.js";
let appId = import.meta.env.VITE_SPARKAI_APPID;
let apiKey = import.meta.env.VITE_SPARKAI_KEY;;
let apiSecret = import.meta.env.VITE_SPARKAI_SECRET;;
let status = "init";
```

杜绝硬编码开发



低耦合代码

服务端响应方案：HTTP轮训、长轮训、SSE、WebSocket



HTTP轮训（polling）

客户端定期向服务器发送HTTP请求，询问是否有新数据。服务器收到请求后，无论是否有数据更新，都会立即响应。

长轮询（Long Polling）

客户端发送HTTP请求到服务器后，服务器会挂起请求，直到有新数据可发送或超时。

服务器发送事件（Server-Sent Events，SSE）

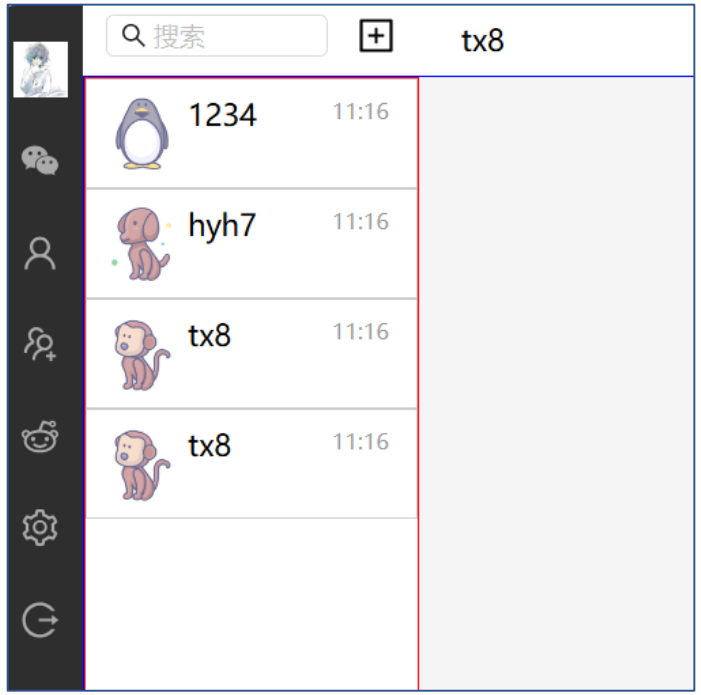
SSE允许服务器向客户端推送信息，而不需要客户端显式请求

WebSocket

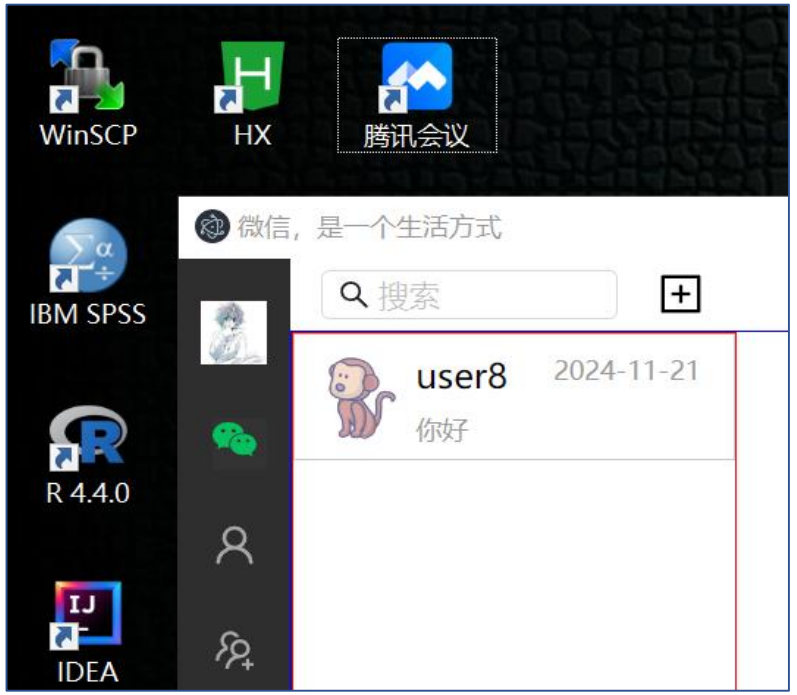
全双工通信通道，客户端和服务器可以在任何时候发送消息，不需要轮询

仿微信，仿的不只是UI

UI与逻辑



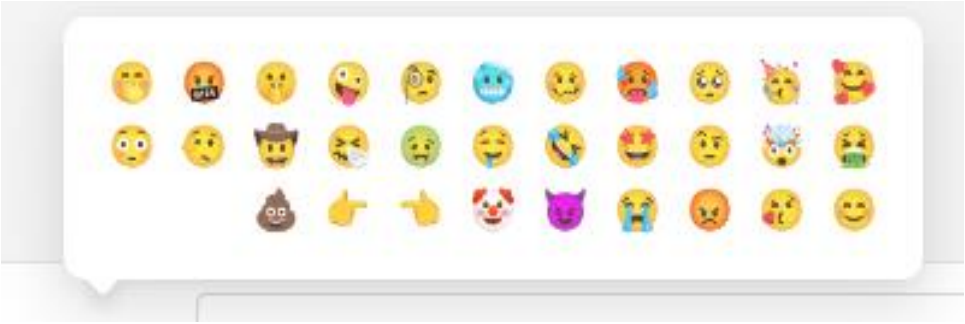
桌面端应用程序支持



功能



支持发送表情



全面超越微信（米式对比法）

对比项	仿微信	微信
AI对话加持	✔	✖
查看每条消息时间	✔	✖
web聊天版	✔	✖
无感知更新	✔	✖
邮箱验证码	✔	✖
发送消息	✔	✔
其他	✖	✔



微信，不可超越的原型

我们小组已知但未实现的细节包括：

聊天模块细节	其他模块
基于UDP协议	公众号生态
对方正在输入中	视频号生态
拍一拍、@功能、表情包	小程序生态
链接识别(内置浏览器引擎)	语音视频通话
黑名单与安全机制	金融模块
多端同步	第三方应用登录支持
文件传输	…
消息转发	
用户未感知	

越开发越觉得，

仿是不可能的。

谁敢说微信是小而美，

我第一个不同意！

尽可能完善自身

WS心跳——确保客户端与服务器保持联系

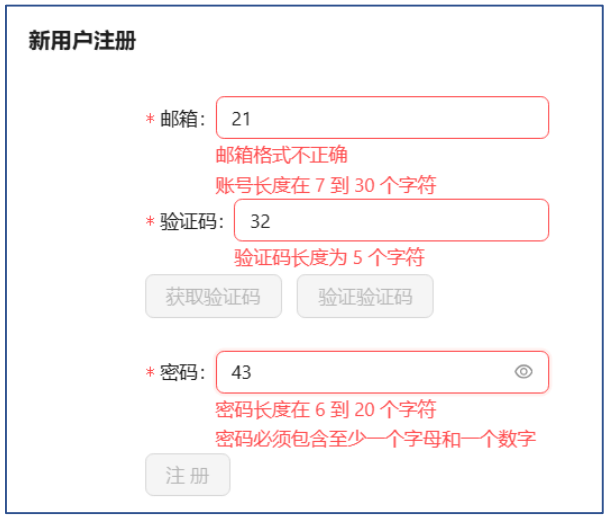
每隔20s客户端发送“ping”，服务端返回“pong”，确保不掉线

```
startHeartBeat() {
    this.heartBeatTimer = setInterval(() => {
        if (this.ws.readyState === WebSocket.OPEN) {
            this.ws.send(`ping-${myUid}`); // 发送ping消息
        } else {
            console.log('WebSocket disconnected. Attemptir
            this.reconnectWebSocket();
        }
    }, 20000); // 每20秒发送一次心跳
},

stopHeartBeat() {
    if (this.heartBeatTimer) {
        clearInterval(this.heartBeatTimer);
        this.heartBeatTimer = null;
    }
},
```

```
public void onMessage(String message) {
    try {
        if (message.startsWith("ping")) {
            String[] parts = message.split( regex: "-");
            String uid = parts[1];// ping-uid
            Session session = sessionPool.get(uid);
            if (session != null && session.isOpen()) {
                session.getBasicRemote().sendText( s: "pong");
            }
            return;
        }
```

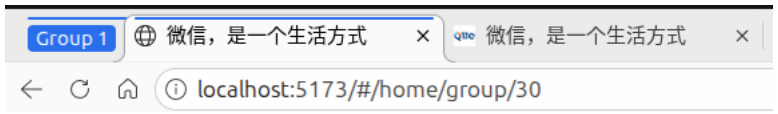
确保细节的完善



输入框验证







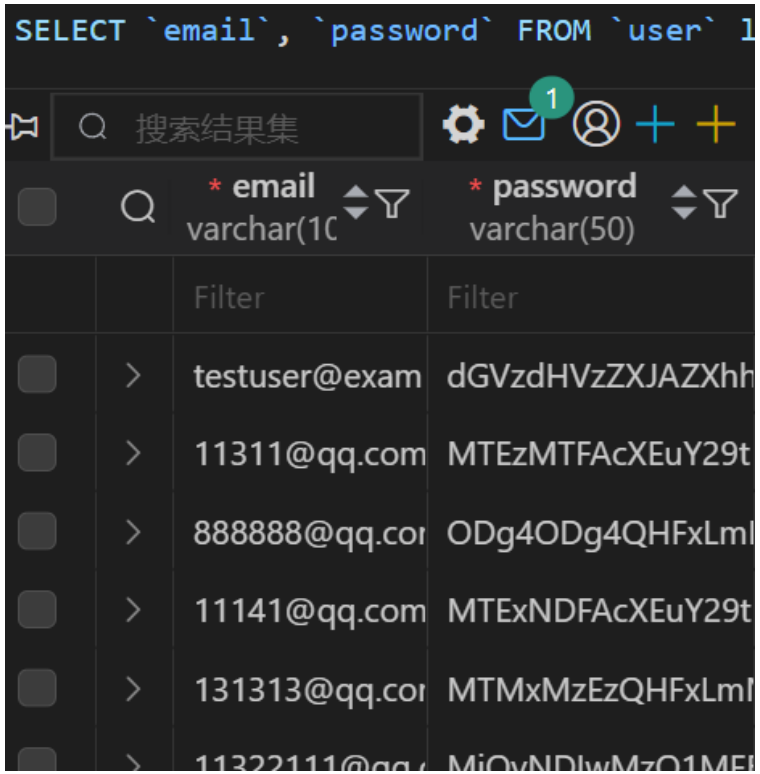
Title



头像点击效果



好友信息展示



用户密码加密

前端性能优化——提供极致响应速度

v-if & v-show 合理使用

```
<div @click="handleChat" class="handle-btn">
    <WechatOutlined :style="{ fontSize: '25px', color: '#576b95' }" />
    <br>
    <span>消息聊天</span>
</div>
<div @click="handleDelete(props.curGroup)" class="handle-btn"
    v-if="props.curGroup?.creatorId == uid">
    <DeleteOutlined :style="{ fontSize: '25px', color: '#576b95' }" />
    <br>
    <span>删除群聊</span>
```

```
<div @contextmenu.prevent="openContextMenu($event)">
    <div class="nick-name">{{ props.nickname }}
        <span v-show="showTime"> | {{ formatTime(props.time) }}</span>
    </div>
    <div class="content">
        <div class="message">{{ props.message }}</div>
        <div class="message" style="border-top: 1px solid #ccc; padding-
            translateText }}</div>
    </div>
```

watch & computed 合理使用

```
const isSelfMessage = computed(() => props.isSelf);
const menuStyle = computed(() => ({
    position: 'fixed',
    left: `${menuPosition.value.x}px`,
    top: `${menuPosition.value.y}px`
}));
```

```
watch(() => user.email, (newV, oldV) => {
  if (newV.length >= 7 && newV.length <= 40) {
    disCodeBtn.value = false  // 启用验证按钮
  } else {
    disCodeBtn.value = true
  }
})

disCodeBtn.value = computed(() => {
  return user.email.length >= 7 && user.email.length <= 40;
});
```

```
63  watch(() => useWs.gid.value, async () => {
64      curGid.value = useWs.gid.value;
65      try {
66          const GidMsgs = await getMsgs({ uid: Number
67  >       GidMsgs.forEach(item => {···
73          });
74          wsStore.setCurHistory(GidMsgs);
75          console.log(useWs.curHistory.value);
76      } catch (error) {
```

前端性能优化——提供极致响应速度

函数防抖

```
// 防抖函数
export const debounce = (func, wait) => {
    let timeout;
    return function (...args) {
        const context = this;
        clearTimeout(timeout);
        timeout = setTimeout(() => func.app
    };
};
```

```
192
193 const _start = () => {···
208 }
209
210 const start = debounce(_start, 500);
211
212 const resend = () => {
213     messages.value.pop();
```

组件懒加载

```
const Login = () => import('@/views/login/Login.vue')
const Home = () => import('@/views/home/Home.vue')
const Message = () => import('@/views/message/Message
const Group = () => import('@/views/message/Group.vue
```

图像懒加载

```
@mouseenter="showTime = true" @mouseleave="showTime = false">
<img class="avatar" v-lazy="props.avatar" alt="avatar" />
<div @contextmenu.prevent="openContextMenu($event)">
    <div class="nick-name">{{ props.nickname }} <span v-show="showTime">
    </div>
    <div class="content">
        <div class="message">{{ props.message }}</div>
        <div class="message" style="border-top: 1px solid #ccc; padding-
            translateText }}</div>
```